

หัวข้อ	รายละเอียด
ชื่องาน	การทดสอบฟีเจอร์ Advanced Table Reservation สำหรับระบบจัดการร้านอาหาร (OMS)
Inventory	Indoor 10 โต๊ะ, Outdoor 5 โต๊ะ
Logic หลัก	การจอง 1 ครั้ง ต้องระบุ Timeslot ละ 1 ชั่วโมงเท่านั้น
Constraint หลัก	หากโต๊ะประเภทนั้นเต็มใน Timeslot ที่เลือก ระบบต้องไม่อนุญาตให้จอง (ป้องกัน Overbooking)
Integration	เมื่อจองสำเร็จ ระบบต้องส่งข้อมูล ไปยัง API ของระบบสมาชิก (CRM) เพื่อสะสมแต้ม

Task 1: Test Strategy & Design — Test Scenarios (High-level)				
No.	ประเภท (Type)	Test Scenario	รายละเอียด / เหตุผลที่ควรทดสอบ	ระดับความสำคัญ (Priority)
1	Happy Path	จองโต๊ะประเภท Indoor สำเร็จเมื่อไม่มีโต๊ะว่างใน Timeslot นั้น	ตรวจสอบ Flow การจองปกติ ตั้งแต่เลือกโต๊ะ รับการจอง จนถึงได้รับการยืนยัน (Confirmation)	High
2	Happy Path	จองโต๊ะประเภท Outdoor สำเร็จเมื่อยังมีโต๊ะว่างใน Timeslot นั้น	ตรวจสอบว่าระบบรองรับโต๊ะทั้ง 2 ประเภทแตกต่างกันอย่างถูกต้อง (Indoor/Outdoor นับ Stock แยกกัน)	High
3	Happy Path	การจองสำเร็จแล้วมีการส่งข้อมูลไปยัง CRM API เพื่อสะสมแต้มสำเร็จ	ตรวจสอบ Integration ระหว่างระบบของกับระบบสมาชิก (CRM) ว่าทำงานถูกต้องเมื่อ Booking สำเร็จ	High
4	Edge Case	จองโต๊ะประเภทที่เต็มแล้วใน Timeslot ที่เลือก (Boundary: โต๊ะสุดท้ายถูกจองไปแล้ว)	ตรวจสอบว่าระบบปฏิเสธการจองอย่างถูกต้องเมื่อจำนวนโต๊ะเต็ม (Indoor=10, Outdoor=5) ไม่เกิด Overbooking	High
5	Edge Case	การจองคาบที่ซ้อนทับ (Overlapping Timeslot) เช่น 001 18:00-19:00 แล้วมีคนที่จอง 01 18:30-19:30 บนโต๊ะประเภทเดียวกัน	ตรวจสอบ Logic การนับ Slot ว่าเช็กลานช่วงเวลาที่กำหนดไว้ได้ทั้งหมดหรือไม่ (จองใหม่จะทับการจองเก่าได้หรือไม่ ขึ้นกับ Slot นามธรรม)	High
6	Edge Case	ยกเลิกการจอง (Cancel) แล้วจองใหม่ทันที (Rebook) ในช่วงเวลาที่ยังไม่ปล่อยโต๊ะ	ตรวจสอบว่าระบบคืนสินค้าได้ (Release Inventory) ทันทีหลัง Cancel และอนุญาตให้ผู้ใช้จองอื่นจองต่อได้โดยไม่มี Stock	High
7	Edge Case	ผู้ใช้ 2 คนพยายามจองโต๊ะตัวสุดท้ายพร้อมกัน (Concurrent Request / Race Condition)	ตรวจสอบว่าระบบมี Locking Mechanism ที่ป้องกันไม่ให้เกิดการจองเกินจำนวนจริงเมื่อมี Request เข้ามาพร้อมกัน	High
8	Edge Case	จองในช่วงเวลาที่ใกล้ถึงรอบ 1 ชั่วโมงเต็มตามที่ระบบกำหนด (เช่น 18:15-19:15 หรือ 18:00-18:30)	ตรวจสอบ Validation 001 Timeslot ว่าระบบอนุญาตเฉพาะช่วงเวลาที่กำหนดไว้ได้ทั้งหมดหรือไม่	Medium
9	Edge Case	การจองสำเร็จในฝั่ง OMS แต่การเรียก CRM API ล้มเหลว (Timeout / CRM Down)	ตรวจสอบว่าระบบจัดการ Transaction อย่างไร (เช่น Retry, Queue, หรือบันทึก Booking สำเร็จแต่ Flag ฝั่งฝั่งเพื่อ Sync ภายหลัง) โดยไม่ทำให้การจองของลูกค้าล้มเหลวไปเลย	High
10	Negative	จองโดยระบุ table_type ที่ไม่มีอยู่ในระบบ (เช่น "VIP")	ตรวจสอบ Input Validation ว่าระบบปฏิเสธค่าที่ไม่อยู่ใน Enum ที่กำหนด (Indoor/Outdoor)	Medium
11	Negative	จองด้วย customer_id ที่ไม่มีอยู่จริงในระบบสมาชิก	ตรวจสอบว่าระบบตรวจสอบความถูกต้องของลูกค้าก่อนสร้าง Booking หรือก่อนเรียก CRM	Medium
12	Edge Case	จองล่วงหน้าเกินวันที่ยังไม่ผ่านแล้ว (Past Date) หรือจองใกล้เกินขอบเขตที่ร้านเปิดรับ (เช่น ภายใน 90 วันล่วงหน้า)	ตรวจสอบ Business Rule เรื่องช่วงวันที่อนุญาตการจอง	Low
13	Edge Case	Timeslot ที่พร้อมที่จะคืน หรือควรมีเวลาปิดโต๊ะแล้ว (เช่น 23:30-00:30)	ตรวจสอบว่าระบบจัดการช่วงเวลาที่ใช้รับบริการเกินเวลาที่กำหนดได้ถูกต้อง หรือปฏิเสธอย่างเหมาะสม	Low

Task 1: Detailed Test Case — Scenario ที่ซับซ้อนที่สุด (Concurrent Booking / Race Condition)	
Test Case ID	TC-007
Scenario อ้างอิง	No. 7 — ผู้ใช้ 2 คนพยายามจองโต๊ะตัวสุดท้ายพร้อมกัน (Concurrent Request / Race Condition)
วัตถุประสงค์ (Objective)	ยืนยันว่าระบบไม่อนุญาตให้เกิดการจองเกินจำนวนโต๊ะที่มีจริง (Overbooking) เมื่อมี Request จะเข้ามาพร้อมกันในช่วงเวลาเดียวกัน สำหรับโต๊ะประเภทเดียวกัน
Preconditions	1. ระบบมีโต๊ะประเภท Indoor ทั้งหมด 10 โต๊ะ 2. วันที่ 2024-05-20 ช่วงเวลา 18:00-19:00 มีโต๊ะ Indoor ถูกจองไปแล้ว 9 โต๊ะ (เหลือว่างอยู่ 1 โต๊ะ) 3. ลูกค้า CUST-001 และ CUST-002 มีข้อมูลถูกต้องในระบบสมาชิก (CRM) แล้วทั้งคู่
Test Data	User A -> customer_id: CUST-001, table_type: Indoor, date: 2024-05-20, timeslot: 18:00-19:00 User B -> customer_id: CUST-002, table_type: Indoor, date: 2024-05-20, timeslot: 18:00-19:00 (ทั้งสอง Request มีค่าตรงกันทุกฟิลด์ ยกเว้น customer_id)
Steps	1. ตั้งค่าข้อมูลเริ่มต้น (Test Data Setup) ให้โต๊ะ Indoor วันที่ 2024-05-20 timeslot 18:00-19:00 เหลือว่างเพียง 1 โต๊ะ 2. ยิง Request POST /api/v1/reservations จาก User A และ User B พร้อมกันแบบขนาน (Concurrent/Parallel Request ห่างกันไม่เกิน 50-100 มิลลิวินาที) เช่น ใช้ postman ยิงพร้อมกัน 2 Thread 3. บันทึก Response (HTTP Status Code + Body) ของทั้งสอง Request 4. ตรวจสอบข้อมูลในตาราง bookings ว่ามีกี่ Record ที่ถูกสร้างสำหรับ Indoor วันที่ 2024-05-20 timeslot 18:00-19:00 5. ตรวจสอบว่าระบบเรียก CRM API เพื่อสะสมแต้มเฉพาะรายการที่จองสำเร็จเท่านั้น (ตรวจสอบ Log การเรียก API หรือ Mock Server) 6. ทำซ้ำการทดสอบ (Repeat) อย่างน้อย 5-10 รอบ เพื่อยืนยันความเสถียรของผลลัพธ์ (Race Condition อาจไม่เกิดทุกครั้งหากไม่มี Lock ที่ดีพอ)
Expected Result	1. มีเพียง 1 ใน 2 Request เท่านั้นที่ได้รับ HTTP 201 Created (จองสำเร็จ) 2. อีก Request ต้องได้รับ HTTP 409 Conflict พร้อมข้อความแจ้งว่าโต๊ะเต็มแล้ว 3. จำนวน Record ในตาราง bookings สำหรับ Indoor/2024-05-20/18:00-19:00 ต้องไม่เกิน 10 รายการ (ไม่เกิด Overbooking) ทุกครั้งที่ทดสอบซ้ำ 4. มีการเรียก CRM API เพื่อสะสมแต้มเพียง 1 ครั้ง ตรงกับลูกค้าที่จองสำเร็จเท่านั้น 5. ไม่มี Error 500 หรือ Deadlock เกิดขึ้นที่ฝั่ง Server ระหว่างการประมวลผลพร้อมกัน
Priority	High
ประเภท (Type)	Functional / Concurrency / Non-happy Path

Task 2: API Testing Analysis — Negative Test Cases (POST /api/v1/reservations)					
No.	Negative Test Case	Expected Request / Input (Invalid)	Expected HTTP Status	Expected Behavior / Message	Remark
1	Missing Payload (Missing Request Body)	ไม่มี customer_id ใน Body ส่วน ("table_type": "Indoor", "date": "2024-05-20", "timestamp": "18:00:19:00")	400 Bad Request	ระบบต้อง Validate ว่าฟิลด์ที่จำเป็นครบถ้วนก่อนประมวลผล และส่ง Error Message ระบุว่าฟิลด์ขาดหายไป	
2	Invalid table_type ที่ไม่อยู่ในรายการ (Invalid Enum Value)	table_type: "XYZ" (ไม่อยู่ในที่ Indoor/Outdoor)	400 Bad Request	ระบบต้องปฏิเสธค่าที่ไม่อยู่ใน whitelist ของระบบและให้ข้อผิดพลาด	
3	รูปแบบวันที่ไม่ถูกต้อง (Invalid Date Format)	date: "20-05-2024" หรือ date: "2024-13-45"	400 Bad Request	ระบบต้อง Validate รูปแบบ YYYY-MM-DD ของค่าวันที่ และแจ้งเตือนถึงข้อผิดพลาด	
4	Timestamp ไม่ตรงกับเวลา : จำลองสถานะระบบที่หมดอายุ	timestamp: "18:00:18:30" หรือ "18:15:19:15"	400 Bad Request	ระบบต้อง Validate Timestamp ของเวลาที่ส่งมาว่าไม่ตรงกับเวลาที่ระบบกำหนด (เช่น 18:00-19:00)	
5	ส่งค่าวันที่เป็นลบ (Past Date)	date: "2023-01-01" (วันที่ก่อนเวลาจริง)	400 Bad Request	ระบบไม่ควรถูกอนุญาตให้จองย้อนหลังได้	
6	customer_id ไม่ได้อยู่ในรายชื่อสมาชิก (CRM)	customer_id: "CUST-00000" (ไม่มีในฐานข้อมูล members)	404 Not Found (หรือ 400 พร้อม error code 00 WUE)	ระบบควรตรวจสอบก่อนดำเนินการจองว่า ID ของลูกค้ามีอยู่ในระบบ CRM หรือไม่	
7	ส่ง Body เป็น Invalid JSON หรือ Empty Body	Invalid หรือ {"table_type": "Indoor"} (Syntax Error)	400 Conflict	ระบบต้องปฏิเสธการ Parsing Error ของข้อมูลและให้ Code ระบุถึงข้อผิดพลาด	Error 400 (Conflict) คือกรณีที่ระบบตรวจสอบพบข้อผิดพลาดทาง HTTP ที่บ่งชี้ว่าข้อมูลไม่ถูกต้องตามที่คาดหวังไว้ อาจเกิดจากข้อมูลรูปแบบไม่ถูกต้อง หรือข้อมูลที่ส่งมาขัดแย้งกับกฎทางธุรกิจของระบบ
		Body: "" หรือ {"table_type": "Indoor"} (Syntax Error)	400 Bad Request	ระบบต้องจัดการ Parsing Error ของข้อมูลและให้ Code ระบุถึงข้อผิดพลาด	

Task 2 (ข้อ 2): สถานะที่เป็นไปไม่ได้เมื่อ API ตอบกลับ 409 Conflict		
No.	สาเหตุที่เป็นไปไม่ได้	รายละเอียด
1	ได้ะประเภทที่ระบุเต็มแล้วใน Timeslot นั้น (Inventory Exhausted)	จำนวนการจองในวัน/เวลาประเภทใดะเดียวกัน ถึงขีดจำกัดสูงสุดแล้ว (Indoor=10, Outdoor=5) ตรงกับ Business Rule หลักของระบบตามโจทย์
2	เกิด Race Condition จากการจองพร้อมกันของผู้ใช้หลายคน (Concurrency Conflict)	มีผู้ใช้ 2 คนขึ้นไปพยายามจองได้ะตัวสุดท้ายในเวลาไล่เลี่ยกัน ระบบตรวจพบว่า Resource ถูกจองไปแล้วโดย Request อื่นก่อนหน้านี้ในช่วงเสี้ยววินาที
3	การจองซ้ำ (Duplicate Booking) จาก Request เดิม	ผู้ใช้งานเดียวกันส่ง Request ซ้ำ (เช่น กด Submit ซ้ำ หรือ Client Retry อัตโนมัติ) สำหรับการจองที่มีอยู่แล้วในระบบ (Idempotency Conflict)
4	สถานะของ Resource เปลี่ยนไปแล้วระหว่างการประมวลผล (State Conflict)	เช่น พยายามจองได้ะที่เพิ่งถูกยกเลิกโดยผู้ระหว่าง Sync สถานะ หรือ Booking ที่ยังอิงผู้ระหว่าง Process อื่น (เช่น ค่าส่งถูกแก้ไข/ยกเลิกพร้อมกัน) ทำให้ข้อมูลขัดแย้งกับสถานะปัจจุบัน

Task 3: Data Integrity & SQL	
1. Verify Stock — คำอธิบาย	เช็คว่าในวันที่ 2024-05-20 เวลา 18:00-19:00 มีโต๊ะประเภท Indoor ถูกจองไปกี่ตัว (จากตาราง bookings) หมายเหตุ: การจองเฉพาะสถานะที่ถือว่า Active อยู่ (เช่น CONFIRMED/BOOKED) เพื่อไม่ให้นับการจองที่ถูกยกเลิก (CANCELLED) รวมเข้าไปด้วย ซึ่งจะทำได้ด้วย Stock ที่ผิดพลาด
1. Verify Stock — SQL Query	SELECT table_type, COUNT(*) AS booked_count FROM bookings WHERE booking_date = '2024-05-20' AND timeslot = '18:00-19:00' AND table_type = 'Indoor' AND status IN ('CONFIRMED', 'BOOKED') GROUP BY table_type;
2. Data Mismatch — คำอธิบาย	พบว่า customer_id ที่มีการจองอยู่ในตาราง bookings แต่ไม่มีข้อมูลอยู่ในตาราง members เพื่อตรวจสอบ Bug ของระบบ Integration (กรณี Booking ถูกสร้างสำเร็จ แต่ข้อมูลลูกค้าไม่ถูกสร้าง/Sync มาถึงระบบสมาชิกอย่างถูกต้อง เช่น กรณี CRM API ส่งผิดพลาดต่อระบบ OMS ไม่ Rollback หรือไม่แจ้งเตือน) ใช้ LEFT JOIN ตรวจสอบด้วย m.customer_id IS NULL เพื่อค้นหาการจองที่ไม่มี Match ในตาราง members
2. Data Mismatch — SQL Query	SELECT DISTINCT b.customer_id FROM bookings b LEFT JOIN members m ON b.customer_id = m.customer_id WHERE m.customer_id IS NULL;
สมมติฐานเกี่ยวกับโครงสร้างตาราง (Assumption)	เนื่องจากโจทย์ไม่ได้ให้ Schema จึงมาจึงสมมติโครงสร้างตารางดังนี้: - ตาราง bookings มีคอลัมน์: booking_id, customer_id, table_type, booking_date, timeslot, status - ตาราง members มีคอลัมน์: customer_id, name, points, ... หากโครงสร้างจริงต่างจากนี้ (เช่น ชื่อคอลัมน์อื่นที่เป็น reservation_date หรือมีตาราง tables แยกต่างหาก) ต้องปรับขึ้นชื่อคอลัมน์ JOIN ให้ตรงกับ Schema จริงก่อนใช้งาน

Task 4: Bug Report — Outdoor Overbooking บน Mobile App	
Bug ID	BUG-2024-0520-01
Title / Summary	[Mobile App] ระบบอนุญาตให้จองได้ประเภท Outdoor เกินจำนวนที่มีจริง (Overbooking) — จองซ้ำหรือซ้อนกับ 6 รายการ ทั้งที่ Stock มีเพียง 5 ได้
Severity	High (กระทบ Core Business Logic โดยตรง ทำให้ร้านรับลูกค้าเกินความจุจริง)
Priority	High (ควรแก้ไขก่อนขึ้น Production)
Environment / Platform	แพลตฟอร์มที่พบปัญหา: Mobile App (iOS/Android) ที่ทดสอบ เช่น iOS 17.x, Android 14 และ App Version x.x.x) แพลตฟอร์มที่ไม่พบปัญหา: Web Application (ทำงานปกติ ปฏิเสธการจองเมื่อได้เต็มจุดจอง)
Preconditions	1. วันและ Timeslot ที่ทดสอบมีได้ Outdoor รวมอยู่พอดี 5 ได้ (เต็มความจุ) 2. ไม่มีผู้ใช้ใดที่จองซ้อนสำหรับการจองอย่างน้อย 6 บัญชี หรือจำลอง Concurrent Request 6 รายการผ่าน Mobile App
Steps to Reproduce	1. เปิด Mobile App และ Login เข้าระบบด้วยบัญชีผู้ใช้ 2. เลือกจองได้ประเภท Outdoor ในวันและ Timeslot ที่มีได้ว่างเหลือพอดี 5 ได้ 3. ทำรายการจอง (Submit) ต่อเนื่อง/พร้อมกัน รวมทั้งหมด 6 ครั้ง สำหรับ Timeslot เดียวกัน ผ่าน Mobile App 4. ตรวจสอบผลลัพธ์การจองทั้ง 6 รายการ และตรวจสอบจำนวนได้ที่ได้ที่ถูกต้องจริงในระบบหลังบ้าน (Database/Admin Panel) 5. ทำการทดสอบซ้ำแบบเดียวกันบนเว็บ (Web) เพื่อเปรียบเทียบผลลัพธ์
Actual Result	ระบบยอมรับการจองซ้ำทั้งหมด 6 รายการผ่าน Mobile App (เกินจากจำนวนได้ Outdoor ที่มีจริงคือ 5 ได้) ทำให้เกิด Overbooking 1 รายการ ในขณะที่การทดสอบบนเว็บกับบน Web ระบบปฏิเสธรายการที่ 6 ได้ตัวอย่างถูกต้อง (409 Conflict)
Expected Result	ระบบควรอนุญาตให้จองได้ Outdoor ได้สูงสุด 5 รายการต่อ Timeslot เท่านั้น และปฏิเสธการจองรายการที่ 6 ด้วยข้อความแจ้งเตือนว่าได้เต็ม (พฤติกรรมควรตรงกับระบบ Mobile App และ Web)
สาเหตุที่คาดว่าน่าเป็นได้ (Suspected Root Cause)	1. Mobile App อาจเรียกใช้ API Endpoint หรือ API Version ที่ละเมิดกับ Web ซึ่งไม่มีการ Validate จำนวนได้คงเหลือที่ Server 2. Mobile App มีการทำ Client-side Caching ของจำนวนได้ว่าง และอนุญาตให้กด Confirm ได้โดยไม่ Refresh ข้อมูลล่าสุดจาก Server ก่อนส่ง Request 3. Business Logic การตรวจสอบ Stock อาจถูก Implement ขึ้นที่ฝั่ง Client (Mobile) แทนที่จะพึ่งพา Server-side Validation เพียงจุดเดียว
ผลกระทบทางธุรกิจ (Business Impact)	ร้านอาจต้องรับมือลูกค้าเกินความจุที่แจ้งจริง ส่งผลต่อประสบการณ์ลูกค้า (Customer Experience) และอาจต้องปฏิเสธลูกค้าที่จองผ่าน App สำหรับส่วนนี้จนกว่าจะตรวจสอบความน่าเชื่อถือของระบบ
สิ่งที่แนบมาด้วย (Attachments)	- Screenshot/Screen Recording ของการจอง 6 รายการบน Mobile App - Log การเรียก API จาก Mobile App (Request/Response ทั้ง 6 รายการ) - ผลลัพธ์จาก Query ข้อมูล (Query จำนวน Booking จริงในตาราง bookings) เปรียบเทียบก่อน-หลัง
Reporter	[ชื่อ QA ผู้รายงาน]
Date Reported	[วันที่พบปัญหา]